

[env] Variables d'environnement – os.environ

DESCRIPTION : Lire, écrire et supprimer les variables d'environnement du processus.
POURQUOI : Configurer des applications, passer des secrets, adapter le comportement.
CONTEXTE : Configuration, CI/CD, secrets, chemins système.

LECTURE :

<code>os.environ</code>	Mapping de toutes les variables (dict-like)
<code>os.environ["PATH"]</code>	Valeur (KeyError si absente)
<code>os.environ.get("HOME")</code>	Valeur ou None si absente
<code>os.environ.get("PORT", "8080")</code>	Valeur ou défaut
<code>os.getenv("HOME")</code>	Alias de <code>environ.get</code>
<code>os.getenv("PORT", "8080")</code>	Alias avec défaut
<code>"HOME" in os.environ</code>	Teste la présence
<code>list(os.environ.keys())</code>	Toutes les clés
<code>dict(os.environ)</code>	Copie en dict standard

ÉCRITURE / SUPPRESSION :

<code>os.environ["MA_VAR"] = "valeur"</code>	Définit (string obligatoire)
<code>os.environ.update({"A": "1", "B": "2"})</code>	Définit plusieurs variables
<code>del os.environ["MA_VAR"]</code>	Supprime (KeyError si absente)
<code>os.environ.pop("MA_VAR", None)</code>	Supprime sans erreur
<code>os.putenv("MA_VAR", "valeur")</code>	Définit (bas niveau, préférer <code>environ</code>)
<code>os.unsetenv("MA_VAR")</code>	Supprime (bas niveau)

EXEMPLES :

```
import os
debug = os.getenv("DEBUG", "false").lower() == "true"
port = int(os.getenv("PORT", "8080"))

# Passer des variables à un sous-processus
import subprocess
env = {**os.environ, "API_KEY": "secret"}
subprocess.run(["mon_script.py"], env=env)
```

[os] Répertoire courant – getcwd / chdir

DESCRIPTION : Lire et modifier le répertoire de travail courant.
POURQUOI : Naviguer dans le système de fichiers depuis Python.

FONCTIONS PRINCIPALES :

<code>os.getcwd()</code>	Répertoire courant (str absolu)
<code>os.getcwdb()</code>	Répertoire courant (bytes)
<code>os.chdir("/nouveau/chemin")</code>	Change le répertoire courant
<code>os.chdir(Path("/nouveau/chemin"))</code>	Accepte aussi un Path

EXEMPLES :

```
print(os.getcwd())           # /home/user/projet
os.chdir("/tmp")
print(os.getcwd())           # /tmp

# Changer temporairement de répertoire
import contextlib
@contextlib.contextmanager
def cd(chemin):
    ancien = os.getcwd()
    os.chdir(chemin)
    try:
        yield
    finally:
        os.chdir(ancien)

with cd("/tmp"):
    print(os.getcwd())        # /tmp
print(os.getcwd())           # revenu au répertoire original
```

[os] Lister les fichiers – listdir / scandir / walk

DESCRIPTION : Lister et parcourir les fichiers et répertoires.
POURQUOI : Exploration du système de fichiers, traitement par lot.
CONTEXTE : Scripts de traitement, indexation, migration de fichiers.

— LISTDIR —

<code>os.listdir()</code>	Liste le répertoire courant
<code>os.listdir("/chemin")</code>	Liste un répertoire (noms seulement)
<code>os.listdir(Path("/chemin"))</code>	Accepte un Path
<code>sorted(os.listdir("/chemin"))</code>	Liste triée

— SCANDIR (RECOMMANDÉ) —

<code>os.scandir()</code>	Itérateur d'entrées du répertoire courant
<code>os.scandir("/chemin")</code>	Itérateur d'entrées (DirEntry)
<code>with os.scandir("/chemin") as it:</code>	Utilisation comme contexte
<code>for entry in it:</code>	
<code>entry.name</code>	Nom du fichier
<code>entry.path</code>	Chemin absolu complet
<code>entry.is_file()</code>	True si fichier ordinaire
<code>entry.is_dir()</code>	True si répertoire
<code>entry.is_symlink()</code>	True si lien symbolique
<code>entry.stat()</code>	Infos stat (taille, dates...)
<code>entry.stat().st_size</code>	Taille en octets
<code>entry.inode()</code>	Numéro d'inode

— WALK – PARCOURS RÉCURSIF —

<code>os.walk("/chemin")</code>	Générateur (racine, dossiers, fichiers)
<code>os.walk("/chemin", topdown=True)</code>	Ordre descendant (défaut)
<code>os.walk("/chemin", topdown=False)</code>	Ordre ascendant (feuilles d'abord)
<code>os.walk("/chemin", followlinks=True)</code>	Suit les liens symboliques
<code>os.walk("/chemin", onerror=handler)</code>	Callback sur erreur

EXEMPLES :

```
# Lister seulement les fichiers .py
fichiers_py = [e.name for e in os.scandir(".")
                if e.is_file() and e.name.endswith(".py")]

# Parcours récursif
for racine, dossiers, fichiers in os.walk("/projet"):
    for f in fichiers:
        chemin_complet = os.path.join(racine, f)
        print(chemin_complet)

# Exclure des dossiers du walk
for racine, dossiers, fichiers in os.walk("/projet"):
    dossiers[:] = [d for d in dossiers if d not in {".git", "__pycache__"}]
    for f in fichiers:
        print(os.path.join(racine, f))

# Taille totale d'un répertoire
total = sum(e.stat().st_size for e in os.scandir(".")
            if e.is_file())
```

[os] Créer et supprimer des répertoires – mkdir / makedirs / rmdir

DESCRIPTION : Créer, supprimer et renommer des répertoires.
POURQUOI : Organiser la structure de fichiers depuis Python.

FONCTIONS PRINCIPALES :

<code>os.mkdir("dossier")</code>	Crée un dossier (erreur si existant)
<code>os.mkdir("dossier", mode=0o755)</code>	Avec permissions Unix
<code>os.makedirs("a/b/c")</code>	Crée les dossiers parents si besoin
<code>os.makedirs("a/b/c", exist_ok=True)</code>	Ne lève pas d'erreur si existant
<code>os.makedirs("a/b/c", mode=0o700)</code>	Avec permissions
<code>os.rmdir("dossier")</code>	Supprime (vide uniquement)
<code>os.removedirs("a/b/c")</code>	Supprime récursivement si vides
<code>os.rename("ancien", "nouveau")</code>	Renomme / déplace
<code>os.rename("a/b", "x/y/z")</code>	Renomme + crée parents si besoin
<code>os.replace("ancien", "nouveau")</code>	Renomme (écrase si existant)

EXEMPLES :

```
os.makedirs("output/logs/2024", exist_ok=True)
```

```
# Supprimer un répertoire non vide
import shutil
shutil.rmtree("dossier_a_supprimer")
```

```
[os] Fichiers – remove / rename / stat / open
```

DESCRIPTION : Supprimer, renommer, inspecter et manipuler des fichiers.
POURQUOI : Opérations de bas niveau sur les fichiers système.

SUPPRESSION :

```
os.remove("fichier.txt")          Supprime un fichier
os.unlink("fichier.txt")          Alias de remove
```

RENOMMAGE :

```
os.rename("old.txt", "new.txt")    Renomme / déplace
os.replace("old.txt", "new.txt")    Renomme (écrase la cible)
```

INFORMATIONS (STAT) :

```
os.stat("fichier.txt")             Infos complètes (os.stat_result)
os.stat("fichier.txt").st_size      Taille en octets
os.stat("fichier.txt").st_mtime     Date de modification (timestamp)
os.stat("fichier.txt").st_atime     Date d'accès
os.stat("fichier.txt").st_ctime     Date de création / changement inode
os.stat("fichier.txt").st_mode      Permissions (mode bits)
os.stat("fichier.txt").st_uid       UID du propriétaire (Unix)
os.stat("fichier.txt").st_gid       GID du groupe (Unix)
os.stat("fichier.txt").st_nlink     Nombre de liens
os.lstat("lien")                   Stat du lien lui-même (pas la cible)
```

DATES LISIBLES :

```
import datetime
ts = os.stat("f.txt").st_mtime
dt = datetime.datetime.fromtimestamp(ts)
```

PERMISSIONS :

```
os.chmod("fichier.txt", 0o644)      Change les permissions Unix
os.chown("fichier.txt", uid, gid)    Change propriétaire (Unix, root)
os.access("fichier.txt", os.R_OK)    Teste lecture
os.access("fichier.txt", os.W_OK)    Teste écriture
os.access("fichier.txt", os.X_OK)    Teste exécution
os.access("fichier.txt", os.F_OK)    Teste existence
```

DESCRIPTEURS DE FICHIER (BAS NIVEAU) :

```
fd = os.open("f.txt", os.O_RDONLY)  Ouvre, retourne un fd
os.read(fd, 1024)                   Lit N octets
os.write(fd, b"données")             Écrit des bytes
os.close(fd)                         Ferme le fd
os.O_RDONLY                          Drapeau lecture seule
os.O_WRONLY                          Écriture seule
os.O_RDWR                            Lecture + écriture
os.O_CREAT                           Créer si absent
os.O_TRUNC                           Tronquer à l'ouverture
os.O_APPEND                           Mode ajout
os.O_EXCL                             Erreur si existant (avec O_CREAT)
```

FICHIERS TEMPORAIRES :

```
os.mkstemp()                         Crée un fichier temp → (fd, chemin)
os.mkstemp(suffix=".txt", dir="/tmp") Avec suffixe et répertoire
os.mkdtemp()                         Crée un répertoire temp → chemin
os.mkdtemp(prefix="mon_", dir="/tmp") Avec préfixe
```

LIENS :

```
os.symlink("cible", "lien")          Crée un lien symbolique
os.link("source", "destination")      Crée un lien physique (hard link)
os.readlink("lien")                  Lit la cible du lien symbolique
os.unlink("lien")                     Supprime un lien symbolique
```

EXEMPLES :

```
# Taille d'un fichier
taille = os.stat("data.csv").st_size
print(f"{taille / 1024:.1f} Ko")
```

```
# Vérifier avant de supprimer
if os.access("temp.txt", os.F_OK):
    os.remove("temp.txt")

# Fichier temporaire sécurisé
import tempfile
fd, chemin = os.mkstemp(suffix=".json")
try:
    os.write(fd, b'{"key": "value"}')
finally:
    os.close(fd)
```

[path] Manipulation de chemins – os.path
--

DESCRIPTION : Construire, décomposer et tester des chemins de fichiers.
 POURQUOI : Portabilité cross-platform (séparateurs / vs \).
 CONTEXTE : Toute manipulation de chemin. Préférer pathlib pour du nouveau code.

— CONSTRUCTION —

os.path.join("a", "b", "c.txt")	"a/b/c.txt" (portable)
os.path.join("/racine", "sous", "f.txt")	"/racine/sous/f.txt"
os.path.abspath("relatif/chemin")	Chemin absolu depuis le cwd
os.path.realpath("chemin/avec/../liens")	Chemin canonique (résout .. et liens)
os.path.expanduser("~/Documents")	Remplace ~ par le home
os.path.expandvars("\$HOME/docs")	Remplace les variables d'env
os.path.normpath("a//b/../c")	Normalise ("a/c")
os.path.normcase("Chemin")	Minuscules sur Windows

— DÉCOMPOSITION —

os.path.dirname("/a/b/c.txt")	"/a/b"
os.path.basename("/a/b/c.txt")	"c.txt"
os.path.split("/a/b/c.txt")	("a/b", "c.txt")
os.path.splitext("fichier.tar.gz")	("fichier.tar", ".gz")
os.path.splitdrive("C:\\Users\\f.txt")	("C:", "\\Users\\f.txt") Windows
os.path.splitroot("/a/b")	("", "/", "a/b") Python 3.12+

— TESTS D'EXISTENCE —

os.path.exists("/chemin")	True si existe (fichier ou dossier)
os.path.isfile("/chemin")	True si fichier ordinaire
os.path.isdir("/chemin")	True si répertoire
os.path.islink("/chemin")	True si lien symbolique
os.path.isabs("/chemin")	True si chemin absolu
os.path.ismount("/chemin")	True si point de montage

— TAILLE ET DATES —

os.path.getsize("/chemin/f.txt")	Taille en octets
os.path.getmtime("/chemin/f.txt")	Date de modification (timestamp float)
os.path.getatime("/chemin/f.txt")	Date d'accès (timestamp float)
os.path.getctime("/chemin/f.txt")	Date de création/changement (timestamp)

— CHEMINS COMMUNS —

os.path.commonpath(["/a/b", "/a/b/c"])	"/a/b"
os.path.commonprefix(["/a/b/c", "/a/b/d"])	"/a/b/" (string, pas chemin)
os.path.relpath("/a/b/c", "/a")	"b/c" (chemin relatif)
os.path.samefile("a.txt", "b.txt")	True si même fichier (inode)

EXEMPLES :

```
# Chemin du script courant
dossier = os.path.dirname(os.path.abspath(__file__))

# Construire des chemins portables
config = os.path.join(os.path.expanduser("~"), ".config", "monapp", "config.json")

# Extension d'un fichier
nom, ext = os.path.splitext("archive.tar.gz")
# nom = "archive.tar", ext = ".gz"

# Chemin relatif entre deux chemins
rel = os.path.relpath("/projet/src/module.py", "/projet")
# "src/module.py"
```

DESCRIPTION : Informations et contrôle des processus système.
POURQUOI : PID courant, signaux, lancement de commandes (legacy).
CONTEXTE : Scripts système, démons, monitoring.

PROCESSUS :

os.getpid()	PID du processus Python courant
os.getppid()	PID du processus parent
os.getpgid(pid)	ID de groupe du processus (Unix)
os.getpgrp()	ID de groupe du processus courant
os.getsid(pid)	ID de session (Unix)
os.kill(pid, signal.SIGTERM)	Envoie un signal à un PID
os.killpg(pgid, signal.SIGTERM)	Envoie un signal à un groupe
os.waitpid(pid, options)	Attend un processus fils
os.wait()	Attend n'importe quel fils (Unix)
os.getlogin()	Nom de l'utilisateur connecté
os.getuid()	UID de l'utilisateur courant (Unix)
os.getgid()	GID du groupe courant (Unix)
os.geteuid()	UID effectif (Unix)
os.getegid()	GID effectif (Unix)
os.getgroups()	Groupes supplémentaires (Unix)

LANCEMENT DE COMMANDES (LEGACY – préférer subprocess) :

os.system("ls -la")	Lance, retourne le code retour
os.popen("ls -la")	Lance, retourne un flux (legacy)
os.popen("ls -la").read()	Lit la sortie

FORK / EXEC (UNIX AVANCÉ) :

os.fork()	Duplique le processus (Unix)
os.execv("/bin/ls", ["/bin/ls", "-la"])	Remplace le processus courant
os.execve("/bin/ls", args, env)	Remplace avec environnement
os.execvp("ls", ["ls", "-la"])	Cherche dans PATH
os.execlp("ls", "ls", "-la")	Variante liste
os._exit(0)	Quitte sans appeler atexit (après fork)
os.abort()	Envoie SIGABRT au processus

EXEMPLES :

```
import os, signal
print(f"Mon PID : {os.getpid()}")

# Envoyer SIGTERM à un processus
os.kill(12345, signal.SIGTERM)

# Vérifier si on est root
if os.getuid() == 0:
    print("Exécuté en tant que root")
```

DESCRIPTION : Informations sur le système d'exploitation et le matériel.
POURQUOI : Adapter le comportement selon la plateforme et les ressources.

FONCTIONS PRINCIPALES :

os.uname()	Infos système (Unix) → sysname, nodename...
os.uname().sysname	"Linux", "Darwin"...
os.uname().nodename	Nom de la machine
os.uname().release	Version du noyau
os.uname().machine	Architecture ("x86_64", "arm64"...)
os.cpu_count()	Nombre de CPUs logiques (ou None)
os.confstr("CS_GNU_LIBC_VERSION")	Infos de configuration système (Unix)
os.sysconf("SC_NPROCESSORS_ONLN")	Paramètre système numérique (Unix)
os.get_terminal_size()	Taille du terminal (colonnes, lignes)
os.get_terminal_size().columns	Nombre de colonnes
os.get_terminal_size().lines	Nombre de lignes
os.getloadavg()	Charge système (1, 5, 15 min) Unix
os.urandom(N)	N octets aléatoires cryptographiques

CONSTANTES SYSTÈME :

os.sep	Séparateur de chemin ("/" ou "\\")
os.altsep	Séparateur alternatif (None ou "/")
os.extsep	Séparateur d'extension (".")

os.pathsep	Séparateur de PATH (":" ou ";")
os.linesep	Fin de ligne ("\n" ou "\r\n")
os.devnull	Chemin de /dev/null
os.curdir	Répertoire courant (".")
os.pardir	Répertoire parent ("..")
os.name	"posix", "nt", "java"

```

EXEMPLES :
print(f"CPUs : {os.cpu_count()}")
print(f"Terminal : {os.get_terminal_size().columns} colonnes")
token = os.urandom(32).hex() # Token sécurisé 64 chars hex

```

[os] Descripteurs de fichiers et pipes – pipe / dup / fdopen

DESCRIPTION : Créer des pipes, dupliquer des descripteurs de fichiers.
POURQUOI : Communication inter-processus, redirection bas niveau.
CONTEXTE : IPC, scripts système, implémentation de shells.

FONCTIONS PRINCIPALES :

os.pipe()	Crée un pipe → (fd_lecture, fd_écriture)
os.pipe2(flags)	Crée un pipe avec drapeaux (Linux)
os.dup(fd)	Duplique un fd (nouveau numéro)
os.dup2(fd, fd2)	Duplique fd vers fd2 (écrase)
os.set_inheritable(fd, bool)	Définit si fd est héritable par fils
os.get_inheritable(fd)	True si fd est héritable
os.fdopen(fd, mode="r")	Enveloppe un fd en objet fichier
os.set_blocking(fd, bool)	Mode bloquant/non-bloquant (Unix)
os.get_blocking(fd)	True si fd est bloquant
os.isatty(fd)	True si fd est un terminal

```

EXEMPLES :
# Créer un pipe simple
r_fd, w_fd = os.pipe()
os.write(w_fd, b"message")
os.close(w_fd)
data = os.read(r_fd, 100)
os.close(r_fd)

# Rediriger stdout vers un fichier (bas niveau)
fd = os.open("sortie.txt", os.O_WRONLY | os.O_CREAT | os.O_TRUNC, 0o644)
os.dup2(fd, 1) # fd 1 = stdout
os.close(fd)

```

[os] Opérations atomiques et avancées – replace / truncate / sendfile

DESCRIPTION : Opérations atomiques et optimisations système.
POURQUOI : Robustesse, performance, opérations sans race condition.

FONCTIONS PRINCIPALES :

os.replace("tmp.txt", "final.txt")	Renommage atomique (POSIX)
os.truncate("f.txt", taille)	Tronque à N octets
os.truncate(fd, taille)	Tronque via fd
os.fsync(fd)	Force l'écriture sur disque
os.fdatasync(fd)	fsync sans métadonnées (Unix)
os.ftruncate(fd, taille)	Tronque via fd (alias)
os.fstat(fd)	stat() via fd
os.fchmod(fd, mode)	chmod via fd (Unix)
os.fchown(fd, uid, gid)	chown via fd (Unix)
os.sendfile(out_fd, in_fd, offset, count)	Copie kernel-space (Unix, très rapide)
os.copy_file_range(src, dst, count)	Copie entre fd (Linux 4.5+)

```

EXEMPLES :
# Écriture atomique (write → tmp → rename)
import tempfile, os
with tempfile.NamedTemporaryFile("w", delete=False, dir=".") as f:
    f.write("contenu important")
    tmp = f.name
os.replace(tmp, "fichier_final.txt") # Atomique sur POSIX

```

DESCRIPTION : Accéder aux répertoires importants du système et de l'utilisateur.
 POURQUOI : Localiser config, données, cache de manière portable.

RÉPERTOIRES COURANTS :

os.path.expanduser("~/")	Répertoire home de l'utilisateur
os.path.expanduser("~/Desktop")	Bureau
os.environ.get("HOME")	Home (Unix)
os.environ.get("USERPROFILE")	Home (Windows)
os.environ.get("APPDATA")	AppData (Windows)
os.environ.get("TEMP") or os.environ.get("TMP")	Répertoire temporaire
os.environ.get("XDG_CONFIG_HOME", os.path.expanduser("~/config"))	Config utilisateur (Linux)
os.environ.get("XDG_DATA_HOME", os.path.expanduser("~/local/share"))	Données utilisateur (Linux)
os.environ.get("XDG_CACHE_HOME", os.path.expanduser("~/cache"))	Cache utilisateur (Linux)

EXEMPLES :

```
import os, sys
home = os.path.expanduser("~")

if sys.platform == "win32":
    config_dir = os.path.join(os.environ["APPDATA"], "monapp")
elif sys.platform == "darwin":
    config_dir = os.path.join(home, "Library", "Application Support", "monapp")
else:
    config_dir = os.path.join(home, ".config", "monapp")

os.makedirs(config_dir, exist_ok=True)
```

=====

RÉCAPITULATIF DU MODULE OS

CATÉGORIE	FONCTION / ATTRIBUT	USAGE
Variables d'env	os.environ / os.getenv()	Lire/écrire l'environnement
Répertoire courant	os.getcwd() / os.chdir()	Lire/changer le cwd
Lister	os.listdir() / os.scandir()	Lister un répertoire
Parcours récursif	os.walk()	Parcourir une arborescence
Créer dossier	os.mkdir() / os.makedirs()	Créer des répertoires
Supprimer dossier	os.rmdir() / os.removedirs()	Supprimer des répertoires
Supprimer fichier	os.remove() / os.unlink()	Supprimer un fichier
Renommer	os.rename() / os.replace()	Renommer / déplacer
Infos fichier	os.stat() / os.path.getsize()	Taille, dates, permissions
Permissions	os.chmod() / os.access()	Modifier/tester permissions
Tests chemin	os.path.exists/isfile/isdir()	Vérifier l'existence
Construire chemin	os.path.join() / os.path.abspath()	Chemins portables
Décomposer chemin	os.path.split() / os.path.splitext()	Nom, dossier, extension
Chemin home	os.path.expanduser("~/")	Répertoire utilisateur
Chemin réel	os.path.realpath()	Résoudre liens et ..
PID	os.getpid() / os.getppid()	PID courant / parent
Signal	os.kill(pid, signal)	Envoyer un signal
CPUs	os.cpu_count()	Nombre de processeurs
Aléatoire crypto	os.urandom(N)	Bytes aléatoires sûrs
Pipe bas niveau	os.pipe()	Créer un pipe (fd)
Constantes	os.sep / os.linesep / os.devnull	Valeurs système portables
Nom OS	os.name	"posix" / "nt"

— RECOMMANDATIONS —

TÂCHE	PRÉFÉRER
Manipulation de chemins	pathlib.Path (plus moderne)
Copier / déplacer des fichiers	shutil.copy / shutil.move
Supprimer un dossier non vide	shutil.rmtree
Lancer des commandes	subprocess.run (pas os.system)
Fichiers temporaires	tempfile.NamedTemporaryFile / mkdtemp
Chercher des fichiers par pattern	glob.glob / pathlib.Path.glob